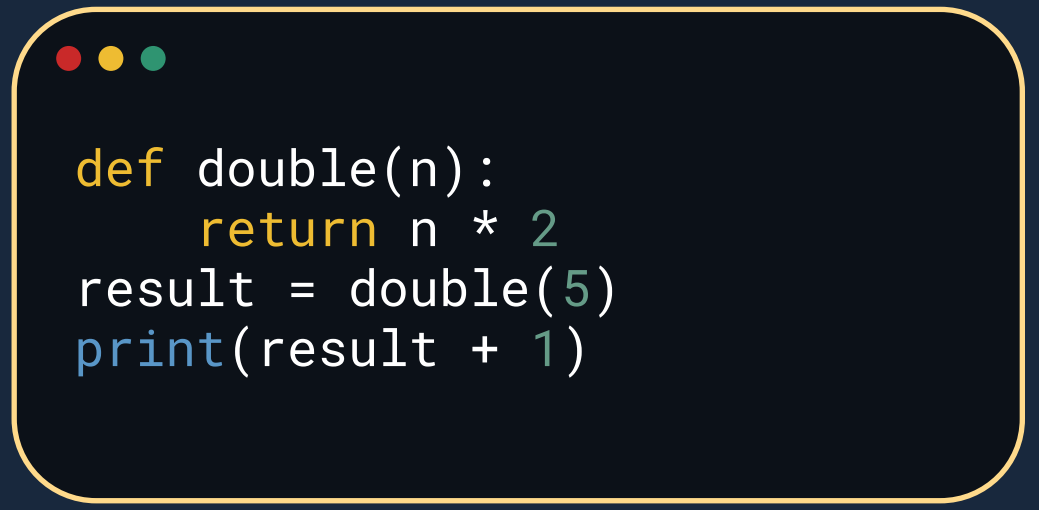
Functions, Parameters & Return Values

Module 1: Python Foundations



```
def double(n):
    return n * 2
result = double(5)
print(result + 1)
```

By the End of Today, You'll Be Able To…

1

Define and call a function

2

Explain why functions beat one long block

You'll Know These by the End of Today

Function

named reusable job

Parameter

input name

Argument

value passed in

Return value

result sent back

CHECK FOR UNDERSTANDING

How many lines print in a 2 × 2 nested loop?

A 2

B 4

C 8

ANSWER KEY

Nested Loop Recall

Correct answer: 4

2 outer passes × 2 inner passes each = 4 total prints. Same multiply-not-add rule from the nesting lesson.

ONE LONG BLOCK

Or One Named, Reusable Piece

Copy-pasting the same loop and conditional gets messy fast. A function names that logic once and reuses it.

A Function Is a Named, Reusable Job

NAME

`def` gives the job a name.

INPUT

Parameters are the ingredients.

OUTPUT

Return value is the finished result.

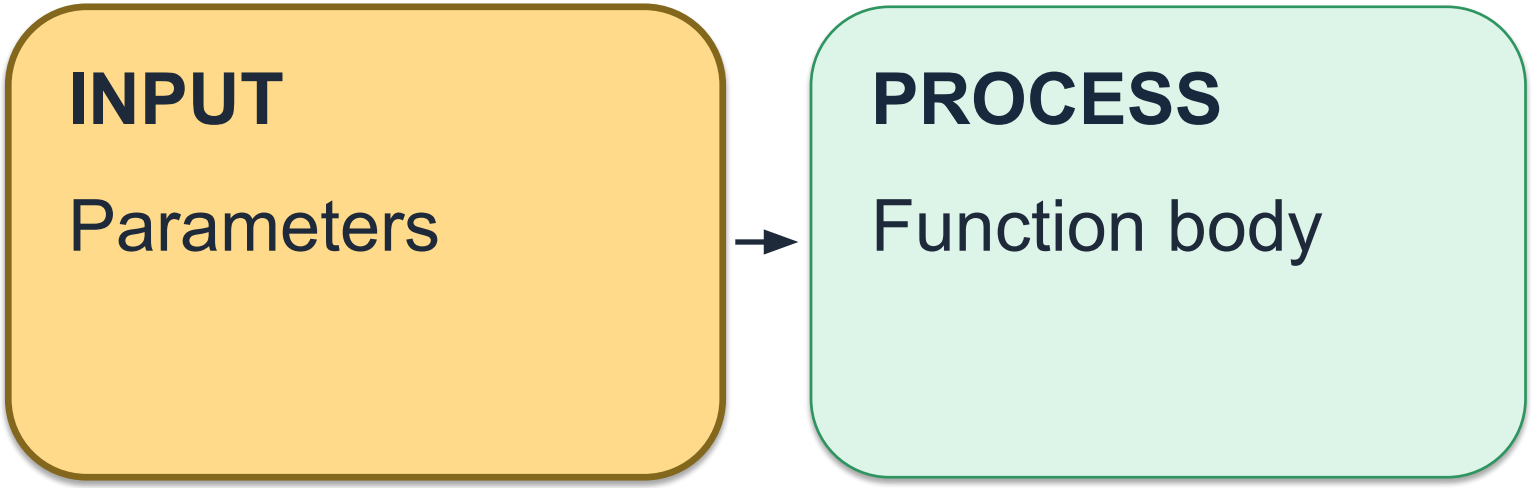
Like a recipe: define it once, then make it again without rewriting every step.

A Function Has a Clear Shape

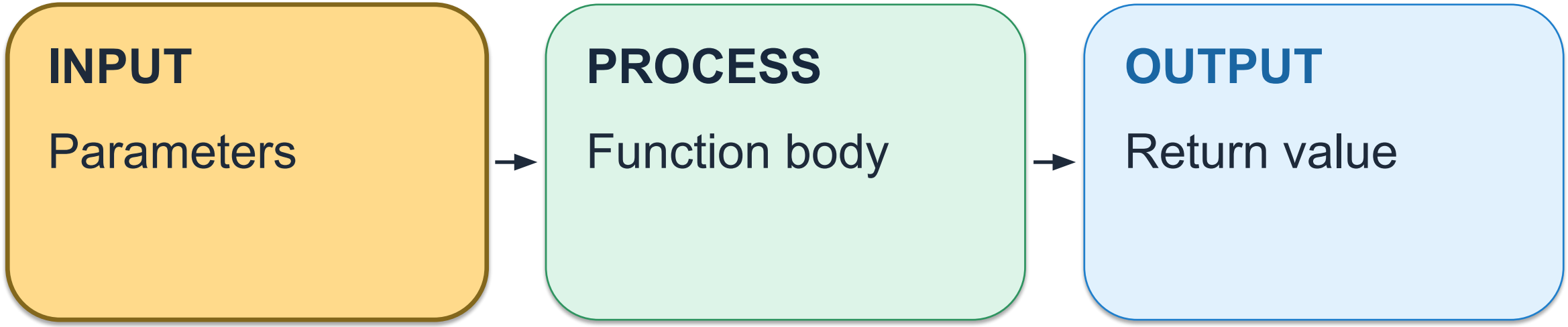
INPUT

Parameters

A Function Has a Clear Shape



A Function Has a Clear Shape



Code Along: Define and Call

A function does nothing until it is actually called. Watch the definition sit still — then run twice.

Define and Call, Live

python

```python
def greet():
    print("Welcome to the Fellowship!")

greet()
greet()
```

ADD TO THE BOARD

Put the Steps of `double(5)` in Order

A

`n` becomes 5

B

the body runs

C

`double` gets called with 5

```
def double(n):
    print(n * 2)
double(5)
```

Parameters Customize It

A recipe that always makes the exact same size cake isn’t very useful. Parameters let the input change each time.

Parameters Customize the Input

PARAMETER

Named in the `def` line — a placeholder.

ARGUMENT

The real value passed in for a call.

Same function. Different input each call.

Code Along: Parameters

Same function, different name each time it’s called — that’s the whole point.

Parameters, Live

```python
def greet(name):
    print("Welcome,", name)

greet("Jordan")
greet("Amara")
```

QUICK CHALLENGE

Given `double(n): print(n * 2)`, then `double(5)`, what prints?

A

5

B

`n * 2`

C

10

ANSWER KEY

Parameter Trace

Correct answer: 10

`n` becomes 5 for this call, so `n` × 2 becomes 10 before `print` runs. `n` is a placeholder for the argument.

Getting a Result Out

Printing shows a value on screen. It does not hand that value back to the caller. `return` does.

Return Sends a Value Back — Print Doesn't

PRINT

Displays a value.
Nothing comes back.

RETURN

Hands a value back to the caller.

A function can do both — or either.

Code Along: Return Values

Watch the difference between displaying a value and actually sending one back for later use.

Return, Live

```python
def double(n):
    return n * 2

result = double(5)
print(result + 1)
```

Watch It Run: Calling `double(5)`

Step 1 of 6

Python learns double. Nothing runs yet

```
def double(n):
    return n * 2
result = double(5)
print(result + 1)
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

QUICK CHALLENGE

A function should return a doubled value, but `total + 1` errors. Most likely cause?

A It prints instead of returning

B Python can’t add 1 to function results

C It needs a second parameter

ANSWER KEY

Return vs. Print Debug

Correct: It prints instead of returning

A function that only prints returns `None`. Storing that call gives `total = None`, and `None + 1` raises `TypeError`.

TURN AND TALK

One Function or Two?

ONE FUNCTION

Calculate and print together.

TWO FUNCTIONS

One calculates; one prints.

Defend your choice.

TIP

“Why Return a Value Instead of Just Printing It?”

PRINTS

```
def double(n):
    print(n * 2)
```

RETURNS

```
def double(n):
    return n * 2
```

Answer like an interview, with PREP: point, reason, example, point.

MODEL ANSWER

Model Answer: PREP

POINT

I return values so other code can use the result.

REASON

`print` only shows a value; the caller gets `None` back.

EXAMPLE

`double(5) + 1` is 11 with `return`, a `TypeError` with `print`.

POINT, RESTATED

So `return` the result; `print` only where it's displayed.

Key Terms — Recap

Function

named reusable job

Parameter

input placeholder

Argument

real input value

Return value

result sent back

So Far, We've Covered…

BEFORE

Loops and conditionals

TODAY

Functions take input and give output

NEXT

Local vs. global variable scope

KEEP LEARNING

Python Tutor — Visualize Function Calls

PRACTICE • 5–10 MIN

Step through a function call, line by line.

Open Python Tutor

Python Docs — Defining Functions